

2장 실습 환경 설정과 파이토치 기초

2.1 파이토치 개요

2.1.1 특징 및 장점

- **GPU 기반 텐서 연산**: NumPy 배열과 유사하나 GPU 가속 지원
- **동적 계산 그래프(define-by-run)**: 코드 실행 시점에 그래프 생성. 조건문·반복문을 파이썬 문법 그대로 사용 가능
- **디버깅 용이성**: 일반 파이썬 디버거(pdb, print) 그대로 적용 가능
- **자동 미분(Autograd)**: 역전파 기울기 자동 계산, 수동 미분 불필요
- **파이썬 친화적 API**: 학습 곡선 완만
- **연구·논문 구현 분야 채택률 높음**

2.1.2 아키텍처 (3계층)

계층	구성 요소	역할
API 계층	<code>torch</code> , <code>torch.nn</code> , <code>torch.optim</code> , <code>torch.autograd</code> , <code>torch.utils.data</code> , <code>torch.onnx</code> , <code>torch.multiprocessing</code>	사용자가 직접 호출하는 파이썬 인터페이스
엔진 계층	Autograd(자동 미분), Aten(C++ 텐서 라이브러리), JIT(연산 최적화 컴파일러)	연산 그래프 구성 및 최적화
연산 처리 계층	TH/THC, THNN/THCUNN, BLAS, CUDA, cuDNN, MKL	CPU/GPU 저수준 텐서 연산 실제 수행

2.2 파이토치 기초 문법

2.2.1 텐서 다루기

텐서 정의

- 데이터를 담은 다차원 배열, 파이토치의 기본 자료 구조
- 0D: 스칼라 / 1D: 벡터 / 2D: 행렬 / 3D 이상: 고차원 텐서

생성 방법

- 직접 생성: `torch.tensor([[1,2],[3,4]])`
- 초기화 생성: `torch.zeros()`, `torch.ones()`, `torch.rand()`, `torch.randn()`, `torch.arange()`
- NumPy 상호 변환: `torch.from_numpy(arr)` ↔ `tensor.numpy()`

데이터 타입

- `torch.FloatTensor` (32비트 실수, 기본), `DoubleTensor` (64비트 실수), `LongTensor` (64비트 정수), `ByteTensor`
- 주의: 손실 함수별 요구 타입 상이. `CrossEntropyLoss` 는 정답 레이블에 Long 타입 요구

GPU 사용

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
tensor = tensor.to(device)
```

- 주의: 연산에 참여하는 모든 텐서와 모델이 **동일 장치**에 존재해야 함

인덱싱·슬라이싱

- NumPy와 동일한 문법 사용 가능

주요 연산

- 사칙 연산: 같은 크기끼리 원소별 연산, 브로드캐스팅 지원
- 행렬 곱: `torch.matmul()`, `torch.mm()`, `@` 연산자

차원 조작 (실무에서 가장 잦은 오류 지점)

- `view()` / `reshape()`: 형태 변경. `view` 는 메모리 연속성 요구, `reshape` 는 자동 처리
- `squeeze()`: 크기 1인 차원 제거 / `unsqueeze(dim)`: 특정 위치에 차원 추가

- `transpose()`, `permute()` : 축 순서 교환
- `cat()` : 기존 차원 방향 연결 / `stack()` : 새 차원 생성하며 쌓기

2.2.2 데이터 준비

세 가지 방식

1. **파일 직접 로드**: pandas로 CSV 읽어 텐서 변환. 소규모 데이터에 적합
2. **커스텀 데이터셋 정의**: `torch.utils.data.Dataset` 상속
 - `__init__` : 데이터 경로·변환 설정
 - `__len__` : 전체 샘플 개수 반환
 - `__getitem__(idx)` : 인덱스에 해당하는 샘플 1건 반환
3. **내장 데이터셋 활용**: `torchvision.datasets` 의 MNIST, FashionMNIST, CIFAR-10 등

DataLoader

- 역할: Dataset을 배치 단위로 묶어 반복 가능한 객체로 변환
- 주요 인자: `batch_size`, `shuffle`, `num_workers` (병렬 로딩), `drop_last` (마지막 불완전 배치 폐기)
- 필요 이유: 전체 데이터를 메모리에 한 번에 적재하지 않고 미니배치 단위 처리 가능

전처리(transforms)

- `transforms.Compose([...])` 로 여러 변환 연결
- `ToTensor()` : PIL/NumPy → 텐서 변환 및 0~1 스케일 조정
- `Normalize(mean, std)` : 채널별 정규화
- `Resize()`, `RandomHorizontalFlip()` 등 증강 기법
- 폴더 구조가 클래스별로 정리된 경우 `ImageFolder` 활용

2.2.3 모델 정의

용어 정리

- 계층(layer): 신경망의 최소 단위. 입력 노드와 출력 노드로 구성
- 블록(block): 계층의 묶음
- 모델(model): 블록·계층이 모여 최종 목적을 수행하는 전체 구조
- 모듈(module): 계층·블록을 포괄하는 파이토치 상위 개념

정의 방식 3가지

1. `nn.Module` 을 상속하지 않는 단순 계층 하나 사용
2. `nn.Sequential` : 순차 구조에 한해 간결하게 표현. `forward` 작성 불필요
3. `nn.Module` 상속(가장 일반적·유연)
 - `__init__` : 사용할 계층을 정의
 - `forward` : 데이터의 흐름(순전파)을 정의

```
class Net(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(784, 128)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(128, 10)
    def forward(self, x):
        return self.fc2(self.relu(self.fc1(x)))
```

주요 계층

- `nn.Linear` (완전 연결), `nn.Conv2d` (합성곱), `nn.MaxPool2d` (풀링), `nn.ReLU` (활성화),
`nn.Dropout` (과적합 방지), `nn.BatchNorm2d` (배치 정규화), `nn.Flatten` (평탄화)

2.2.4 모델의 파라미터 정의

손실 함수 (문제 유형별 선택)

- 회귀: `nn.MSELoss()` — 평균 제곱 오차

- 이진 분류: `nn.BCELoss()` (Sigmoid 출력 전제)
- 다중 분류: `nn.CrossEntropyLoss()` — 내부에 Softmax 포함, 따라서 모델 마지막에 Softmax 중 복 적용 불필요

옵티마이저

- `optim.SGD`: 기본 경사 하강법, 모멘텀 옵션
- `optim.Adam`: 적응적 학습률, 실무 기본 선택지
- `optim.RMSprop`, `optim.Adagrad` 등
- 핵심 메서드: `zero_grad()` (기울기 초기화), `step()` (파라미터 갱신)

학습률 스케줄러

- `torch.optim.lr_scheduler`의 `StepLR`, `ReduceLR0nPlateau`, `LambdaLR` 등
- 목적: 학습 진행에 따라 학습률을 점진적으로 감소시켜 수렴 안정화

용어 구분 (혼동 빈발)

- 에포크(epoch): 전체 훈련 데이터를 1회 순회
- 배치(batch): 한 번에 처리하는 샘플 묶음
- 스텝/이터레이션: 배치 1개 처리 = 파라미터 1회 갱신
- 관계: 1 에포크의 스텝 수 = 전체 데이터 수 ÷ 배치 크기

2.2.5 모델 훈련

표준 학습 루프

```
for epoch in range(num_epochs):
    model.train()
    for x, y in train_loader:
        x, y = x.to(device), y.to(device)
        optimizer.zero_grad()      # ① 기울기 초기화
        output = model(x)          # ② 순전파
```

```

loss = criterion(output, y) # ③ 손실 계산
loss.backward()             # ④ 역전파
optimizer.step()            # ⑤ 파라미터 갱신

```

주의 사항

- `zero_grad()` 필수: 파이토치는 기울기를 누적하므로 초기화 없으면 이전 배치 기울기가 섞임
- `model.train()`: 드롭아웃/배치 정규화를 훈련 모드로 전환

2.2.6 훈련 평가

평가 루프

```

model.eval()
with torch.no_grad():
    for x, y in test_loader:
        output = model(x)
        pred = output.argmax(dim=1)

```

핵심 2요소

- `model.eval()`: 드롭아웃 비활성화, 배치 정규화가 학습된 이동 평균 사용
- `torch.no_grad()`: 기울기 계산 그래프 미생성 → 메모리 절약·속도 향상
- 두 가지는 역할이 다르므로 **함께 사용** 필요

평가 지표

- 정확도(accuracy) 기본, 클래스 불균형 시 정밀도·재현율·F1 병행 고려

2.2.7 훈련 과정 모니터링

텐서보드 활용

```
from torch.utils.tensorboard import SummaryWriter
writer = SummaryWriter("runs/exp1")
writer.add_scalar("Loss/train", loss.item(), epoch)
writer.close()
```

- 실행: 터미널에서 `tensorboard --logdir=runs`
- `add_scalar` (손실·정확도 추이), `add_graph` (모델 구조), `add_image` (입력 샘플) 등 제공
- 목적: 손실 곡선을 통한 과적합/과소적합 조기 판별

모델 저장 및 불러오기

- 권장 방식: `torch.save(model.state_dict(), "model.pt")` → `model.load_state_dict(torch.load("model.pt"))`
- 이유: 모델 전체 저장보다 이식성·안정성 우수

2.3 실습 환경 설정

2.3.1 아나콘다 설치

- 파이썬 배포판 + 패키지 관리자 + 가상 환경 관리 통합 제공
- 프로젝트별 라이브러리 버전 충돌 방지 목적

2.3.2 가상 환경 생성 및 파이토치 설치

```
conda create -n torch_book python=3.9
conda activate torch_book
```

- GPU 사용 시: 그래픽카드 → CUDA → cuDNN → 파이토치 버전의 **호환성 확인 필수**
- 공식 사이트(pytorch.org)에서 OS·패키지 관리자·CUDA 버전 선택 후 제공되는 설치 명령 사용
- 설치 검증: `torch.__version__`, `torch.cuda.is_available()` 확인

- 주피터 노트북 사용 시 커널 등록 별도 수행

2.4 파이토치 코드 맛보기

1. 범주형 변수 중심의 표 형태 데이터(자동차 평가 데이터) 로드
2. 범주형 → 수치형 인코딩, 임베딩 크기 산정
3. 훈련/테스트 분할 및 텐서 변환
4. `nn.Module` 상속 모델 정의(임베딩 + 완전 연결 계층 + 드롭아웃)
5. 손실 함수·옵티마이저 지정 후 학습 루프 실행
6. 에포크별 손실 출력 및 테스트 데이터 평가